🧠 Code Archaeology

(Arqueologia de código — investigar o passado do projeto)

👉 Aqui você aprende a responder perguntas como:

- “Quem fez essa alteração?”
- “Quando isso mudou?”
- “Por que esse bug apareceu?”
- “Onde isso foi introduzido?”

🧬 Ideia central

Git não é só para salvar código

É para **investigar a história do código**

🔍 1. `git log` — ver o histórico

📌 Comando:

```
git log
```

🧠 O que mostra?

- lista de commits
- autor
- data
- mensagem

🔧 Exemplo

```
commit a1b2c3
Author: João
Date: hoje

"corrige bug no login"
```

⚡ Versão simples

```
git log --oneline
```

🔧 Exemplo

```
a1b2c3 corrige bug
d4e5f6 adiciona login
```

💡 Uso real

👉 ver rapidamente o histórico

🌳 2. `git log --graph` — visualizar estrutura

📌 Comando:

```
git log --graph
```

🧠 O que mostra?

👉 as branches e merges visualmente

🔧 Exemplo

```
* commit D
|\
| * commit C
* | commit B
|/
* commit A
```

💡 Uso

👉 entender como branches foram unidas

📂 3. `git log <arquivo>` — histórico de um arquivo

📌 Comando:

```
git log app.txt
```

🧠 O que faz?

👉 mostra todos os commits que alteraram o arquivo

💡 Uso

👉 investigar mudanças específicas

🔄 4. `git log --follow` — acompanhar renomeações

📌 Comando:

```
git log --follow app.txt
```

🧠 O que faz?

👉 mostra histórico mesmo se o arquivo mudou de nome

🔎 5. `git log -G` — buscar mudanças no código

📌 Comando:

```
git log -G "preço"
```

🧠 O que faz?

👉 encontra commits que:

- adicionaram
- removeram

essa palavra

🔥 Exemplo real

👉 “quando mexeram no preço?”

🕵️ 6. `git blame` — quem fez isso?

📌 Comando:

```
git blame app.txt
```

🧠 O que mostra?

Para cada linha:

- quem escreveu
- quando
- qual commit

🔧 Exemplo

```
a1b2c3 (João) preço = 20
d4e5f6 (Maria) total = 50
```

💡 Uso real

👉 descobrir quem introduziu um bug

🧠 7. Fluxo de investigação (nível profissional)

🔥 Cenário: apareceu um bug

1. Ver histórico:

```
git log --oneline
```

2. Comparar versões:

```
git diff commit_antigo commit_novo
```

3. Procurar mudança específica:

```
git log -G "erro"
```

4. Descobrir autor:

```
git blame arquivo.txt
```

Exemplo completo

Você tem:

```
preço = 10 → virou 999 (erro)
```

Investigação:

```
git log -G "preço"
```

↓

```
git show commit_suspeito
```

↓

```
git blame app.txt
```

Resultado:

👉 você descobre:

- quem mudou
- quando mudou
- o que mudou

⚠️ Erros comuns

❌ não usar log

→ fica perdido

❌ não investigar histórico

→ não entende bugs

❌ culpar sem analisar

→ erro de julgamento

🧠 Mentalidade de engenheiro

Você não “adivinha” bugs.

Você:

- investiga
- analisa histórico
- encontra causa

🧭 Regra de ouro

Todo bug tem uma origem — o Git te mostra ela

🚀 Parabéns — você fechou o ciclo do Git

Você aprendeu:

- iniciar projetos
- commits
- branches
- merge vs rebase
- controle total (reset, stash)
- diff
- histórico
- investigação profunda

👉 Isso já é nível profissional/intermediário avançado

💧 Próximo passo (muito importante)

Agora você pode:

1. 🔧 Praticar em projeto real

(ex: seu sistema de delivery)

2. 🌍 Usar com GitHub

(push, pull, colaboração)

3. 🧠 Evoluir para fluxo profissional:

- Git Flow
- Pull Requests
- Code Review